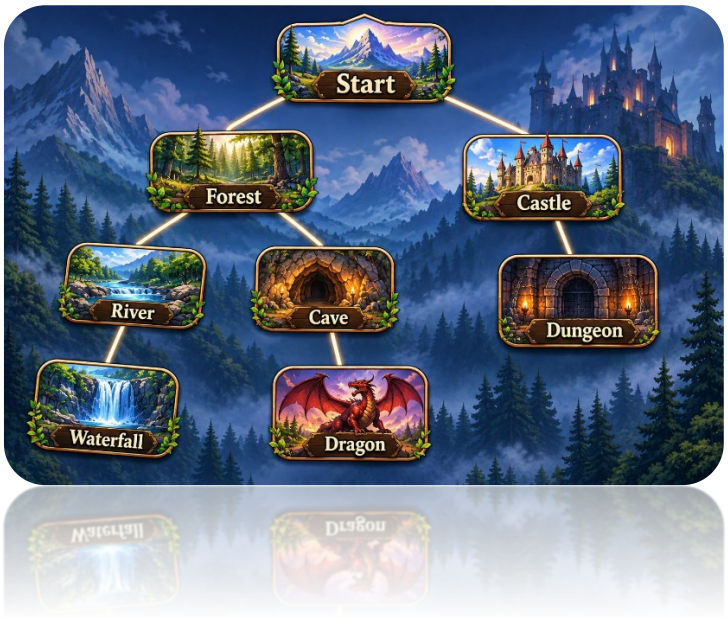
سؤال یک – ۱۰ امتیاز

مرحله‌ی مخفی بازی

در یک بازی ماجراجویی، مراحل بازی به شکل یک درخت طراحی شده‌اند. هر مرحله می‌تواند چند مرحله‌ی بعدی داشته باشد و بازیکن پس از تمام‌کردن یک مرحله، وارد یکی از مراحل فرزند آن می‌شود.

ساختار مراحل بازی به صورت زیر است:



در این ساختار، Start مرحله‌ی آغازین بازی است.

به سؤال‌های زیر پاسخ دهید:

الف) پدر مرحله‌ی Dragon کدام مرحله است؟

ب) عمق مرحله‌ی Dragon را به دست آورید. عمق Startبرابر ۰ در نظر گرفته می‌شود.

ج) ارتفاع این درخت را بر اساس بیشترین عمق موجود در درخت به دست آورید.

د) اگر مراحل با پیمایش Preorder بررسی شوند، مراحل به چه ترتیبی دیده می‌شوند؟

سؤال دو — ۳۰ امتیاز

بازسازی نقشه یک موزه

در یک موزه‌ی بزرگ، اتاق‌ها به‌صورت یک درخت دودویی به هم متصل شده‌اند. هر اتاق یک شماره‌ی یکتا دارد و هر اتاق حداکثر دو اتاق دیگر را به‌عنوان اتاق فرعی خود دارد.

مدیر موزه نقشه‌ی اصلی را در اختیار ندارد، اما دو گزارش از بازدیدهای قبلی باقی مانده است:

- گزارش اول، ترتیب بازدید اتاق‌ها به روش Preorder است .
- گزارش دوم، ترتیب بازدید همان اتاق‌ها به روش Inorder است .

از آنجا که شماره‌ی همه‌ی اتاق‌ها یکتا است، این دو گزارش برای بازسازی ساختار کامل درخت کافی هستند.

برنامه‌ای بنویسید که با دریافت این دو ترتیب، ساختار درخت را بازسازی کند و ترتیب Postorder اتاق‌ها را چاپ کند.

ورودی

در خط اول عدد n، تعداد اتاق‌های موزه، داده می‌شود.

در خط دوم، n عدد صحیح یکتا داده می‌شود که ترتیب Preorder اتاق‌ها را نشان می‌دهد.

در خط سوم، n عدد صحیح یکتا داده می‌شود که ترتیب Inorder همان اتاق‌ها را نشان می‌دهد.

تضمین می‌شود که:

- شماره‌ی اتاق‌ها یکتا هستند .
- هر دو ترتیب مربوط به یک درخت دودویی معتبر هستند .
- n حداقل ۱ است .

$1 \le n \le 1000$

$1 \le room_number \le 10^9$

خروجی

ترتیب Postorder اتاق‌ها را در یک خط و با یک فاصله بین شماره‌ها چاپ کنید.

نمونه ورودی:

```
7
1 2 4 5 3 6 7
4 2 5 1 6 3 7
```

نمونه خروجی:

```
4 5 2 6 7 3 1
```

سؤال سه — ۵۰ امتیاز

مدیریت بسته‌های انبار فروشگاه

یک فروشگاه اینترنتی بزرگ، برای مدیریت بسته‌های مشتریان، به هر بسته یک کد عددی یکتا اختصاص می‌دهد. این کدها در یک درخت جستجوی دودویی (BST) نگه‌داری می‌شوند.

درBST ، کدهای کوچک‌تر از هر گره در زیردرخت چپ و کدهای بزرگ‌تر در زیردرخت راست آن قرار می‌گیرند. اگر کدی از قبل در درخت وجود داشته باشد، دوباره اضافه نمی‌شود.

برنامه‌ای بنویسید که عملیات‌های زیر را روی BST انجام دهد:

- ADD X: بسته‌ای با کد x را اضافه کنید. اگر x از قبل وجود داشته باشد، تغییری ایجاد نشود.
- REMOVE X: بسته‌ای با کد x را حذف کنید. اگر وجود نداشته باشد، تغییری ایجاد نشود.

- FIND X: اگر x در درخت وجود داشت YES و در غیر این صورت NOچاپ کنید.
- PRINT: کد تمام بسته‌ها را با پیمایش Inorder و به ترتیب صعودی چاپ کنید. اگر درخت خالی بود، یک خط خالی چاپ شود.

در حذف یک گره، هر سه حالت گره بدون فرزند، گره با یک فرزند و گره با دو فرزند را به‌درستی مدیریت کنید.

ورودی

در خط اول عدد q، تعداد عملیات‌ها، داده می‌شود.

در q خط بعدی یکی از چهار دستور زیر قرار دارد:

ADD X
REMOVE X
FIND X
PRINT

$1 \le q \le 2000$

$1 \le x \le 10^9$

خروجی

برای هر دستورFIND ، نتیجه‌ی جستجو (YES یا NO) را در یک خط چاپ کنید.

برای هر دستورPRINT ، تمام کدهای موجود در درخت را به ترتیب صعودی و با یک فاصله بین آن‌ها چاپ کنید.

برای دستورات ADD و REMOVE خروجی‌ای چاپ نکنید.

ورودی:

```
12
ADD 50
ADD 30
ADD 70
ADD 20
ADD 40
ADD 60
ADD 80
FIND 40
REMOVE 30
PRINT
FIND 30
PRINT
```

خروجی:

```
YES
20 40 50 60 70 80
NO
20 40 50 60 70 80
```

سؤال چهار — ۷۰ امتیاز

ملاقات دو مسافر در مترو

در یک شهر بزرگ، ایستگاه‌های مترو با استفاده از یک درخت جستجوی دودویی (BST) مدیریت می‌شوند. هر ایستگاه یک کد عددی یکتا دارد.

ابتدا کد ایستگاه‌ها به همان ترتیبی که در ورودی داده می‌شوند، داخل BSTقرار می‌گیرند.

دو مسافر در دو ایستگاه a و b قرار دارند. برای هر دو ایستگاه باید نزدیک‌ترین جد مشترک آن‌ها

(Lowest Common Ancestor یا LCA) را پیدا کنید.

اگر یکی از دو ایستگاه خودش جد دیگری باشد، همان ایستگاه به‌عنوان LCA در نظر گرفته می‌شود.

اگر حداقل یکی از دو ایستگاه در BST وجود نداشته باشد، باید NOT FOUNDچاپ شود.

برای پیدا کردن LCA نباید کل درخت را برای هر درخواست پیمایش کنید؛ از ویژگی BST استفاده کنید.

ورودی

در خط اول عدد n، تعداد ایستگاه‌ها، داده می‌شود.

در خط دوم n عدد صحیح یکتا داده می‌شود که کد ایستگاه‌ها هستند. این اعداد باید دقیقاً به همان ترتیب داخل BST درج شوند.

در خط سوم عدد q، تعداد درخواست‌ها، داده می‌شود.

در q خط بعدی، در هر خط دو عدد a و b داده می‌شود که کد دو ایستگاه موردنظر هستند.

محدودیت‌ها:

$1 \le n \le 2000$

$1 \le q \le 2000$

$1 \le a, b \le 10^9$

خروجی

برای هر درخواست، اگر هر دو ایستگاه در BST وجود داشته باشند، کد نزدیک‌ترین جد مشترک آن‌ها را در یک خط چاپ کنید؛ در غیر این صورت NOT FOUND چاپ کنید.

نمونه ورودی:

```
9
50 30 80 20 40 70 90 35 10
5
35 40
35 70
20 10
50 90
35 100
```

نمونه خروجی:

```
40
50
20
50
NOT FOUND
```

سؤال پنجم — ۱۰۰ امتیاز

سیستم تحلیل صرافی ارز دیجیتال

در یک صرافی ارز دیجیتال، سفارش‌های خرید کاربران دائماً ثبت یا حذف می‌شوند. برای تحلیل سریع بازار، قیمت سفارش‌ها در یک درخت جستجوی دودویی پیشرفته (Augmented BST) نگهداری می‌شوند.

ممکن است چند سفارش قیمت یکسانی داشته باشند. برای هر قیمت فقط یک گره در درخت وجود دارد و تعداد سفارش‌های آن قیمت در count نگهداری می‌شود.

هر گره علاوه بر price وcount ، دو مقدار زیر را نیز نگه می‌دارد:

- size: تعداد کل سفارش‌های موجود در آن گره و زیردرخت‌های آن
- sum: مجموع قیمت تمام سفارش‌های موجود در آن گره و زیردرخت‌های آن

این دو مقدار باید بعد از هر ثبت یا حذف سفارش به‌درستی به‌روزرسانی شوند

استفاده از مرتب‌سازی آماده یا ذخیره‌کردن سفارش‌ها در آرایه و مرتب‌کردن آن‌ها مجاز نیست.

عملیات‌ها:

1 price

یک سفارش با قیمت price ثبت کنید. اگر این قیمت از قبل وجود داشته باشد، فقط count آن را یک واحد افزایش دهید.

2 price

یک سفارش با قیمت price حذف کنید. اگر count بزرگ‌تر از ۱ باشد، فقط count را کاهش دهید؛ در غیر این صورت گره مربوط به آن قیمت را از BST حذف کنید. اگر چنین قیمتی وجود نداشته باشد، تغییری ایجاد نمی‌شود.

3 L R

مجموع قیمت سفارش‌هایی را که قیمتشان در بازه‌ی بسته‌ی [L, R] قرار دارد، چاپ کنید.

4 k

قیمت kاُمین سفارش گران‌تر را چاپ کنید. سفارش‌هایی با قیمت یکسان، در رتبه‌بندی جداگانه حساب می‌شوند. اگر تعداد سفارش‌ها کمتر از k باشد، چیزی چاپ نکنید.

5 k

میانگین قیمت گران‌ترین k سفارش را با تقسیم صحیح محاسبه و چاپ کنید. اگر تعداد سفارش‌ها کمتر از k باشد، چیزی چاپ نکنید.

ورودی

در خط اول عدد q، تعداد عملیات‌ها، داده می‌شود

در qخط بعدی یکی از عملیات‌های زیر قرار دارد:

1 price
2 price
3 L R
4 k
5 k

محدودیت‌ها:

$1 \le q \le 2000$
$1 \le price, L, R \le 10^9$
$L \le R$
$1 \le k \le 100000$

خروجی

برای هر عملیات نوع ۳، ۴ و ۵، نتیجه را در یک خط چاپ کنید.

برای عملیات نوع ۱ و ۲ هیچ خروجی‌ای چاپ نمی‌شود.

نمونه ورودی:

```
12
1 50000
1 45000
1 60000
1 45000
3 45000 60000
4 2
5 2
2 45000
3 40000 70000
4 3
5 3
4 1
```

نمونه خروجی :

```
200000
50000
55000
155000
45000
51666
60000
```